

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5113

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

*I **Kartheeka Repalle (192372289)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kartheeka Repalle

Annexure A

Experiments

- 1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.**

```
import hashlib
import time

messages = [
    "Hello World",
    "Cyber Security",
    "Python Cryptography",
    "Digital Security",
    "This is a test message"
]

print("MD5 vs SHA-256 HASH COMPARISON")
print("=" * 70)

for message in messages:
    print("\nMessage:", message)

    # MD5
    start = time.perf_counter()
    md5_hash = hashlib.md5(message.encode()).hexdigest()
    md5_time = time.perf_counter() - start

    # SHA-256
    start = time.perf_counter()
    sha256_hash = hashlib.sha256(message.encode()).hexdigest()
    sha256_time = time.perf_counter() - start

    print("MD5 Hash    :", md5_hash)
    print("SHA-256 Hash :", sha256_hash)
```

```
print("MD5 Time    :", md5_time)
print("SHA-256 Time :", sha256_time)

print("\nANALYSIS")
print("-" * 70)
print("1. MD5 produces a 128-bit (32 hexadecimal characters) hash.")
print("2. SHA-256 produces a 256-bit (64 hexadecimal characters) hash.")
print("3. MD5 is faster in many situations but is cryptographically broken.")
print("4. MD5 is vulnerable to practical collision attacks.")
print("5. SHA-256 provides significantly stronger collision resistance.")
print("6. SHA-256 should be preferred for modern security applications.")
```

Output:

MD5 vs SHA-256 HASH COMPARISON

=====

=

Message: Hello World

MD5 Hash : b10a8db164e0754105b7a99be72e3fe5

SHA-256 Hash : a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b...

MD5 Time : 0.00000...

SHA-256 Time : 0.00000...

Message: Cyber Security

MD5 Hash : ...

SHA-256 Hash : ...

ANALYSIS

1. MD5 produces a 128-bit (32 hexadecimal characters) hash.
2. SHA-256 produces a 256-bit (64 hexadecimal characters) hash.
3. MD5 is faster in many situations but is cryptographically broken.
4. MD5 is vulnerable to practical collision attacks.
5. SHA-256 provides significantly stronger collision resistance.
6. SHA-256 should be preferred for modern security applications.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

```
# RSA Digital Signature using Python
# Requires: pip install cryptography

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Step 1: Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Generate public key
public_key = private_key.public_key()

# Step 2: Message to be signed
message = b"This is a confidential message."

print("RSA DIGITAL SIGNATURE")
print("=" * 60)

print("\nOriginal Message:")
print(message.decode())

# Step 3: Create digital signature
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
```

```

        hashes.SHA256()
    )

    print("\nDigital Signature:")
    print(signature.hex())

# Step 4: Verify the signature
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\nSignature Verification: SUCCESS")
    print("The message is authentic and has not been modified.")

except Exception:
    print("\nSignature Verification: FAILED")

# Step 5: Test message modification
modified_message = b"This is a modified message."

try:
    public_key.verify(
        signature,
        modified_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

```

```

print("\nModified Message Verification: SUCCESS")

except Exception:
    print("\nModified Message Verification: FAILED")
    print("The message has been modified or the signature is invalid.")

print("\nANALYSIS")
print("-" * 60)
print("1. RSA uses a private key for signing and a public key for verification.")
print("2. The digital signature provides message integrity.")
print("3. Signature verification helps establish authenticity.")
print("4. Modification of the message causes signature verification to fail.")
print("5. RSA-PSS with SHA-256 provides stronger modern signature security.")

```

Output:

RSA DIGITAL SIGNATURE

=====

Original Message:

This is a confidential message.

Digital Signature:

8a31f7c2.....

Signature Verification: SUCCESS

The message is authentic and has not been modified.

Modified Message Verification: FAILED

The message has been modified or the signature is invalid.

ANALYSIS

1. RSA uses a private key for signing and a public key for verification.
2. The digital signature provides message integrity.

3. Signature verification helps establish authenticity.
4. Modification of the message causes signature verification to fail.
5. RSA-PSS with SHA-256 provides stronger modern signature security.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

HMAC (Hash-based Message Authentication Code) using Python

No external library is required.

```
import hashlib
```

```
import hmac
```

```
import time
```

```
message = b"This is an important message."
```

```
secret_key = b"my-secret-key"
```

```
print("HMAC IMPLEMENTATION")
```

```
print("=" * 60)
```

```
# -----
```

```
# 1. Simple SHA-256 Hash
```

```
# -----
```

```
simple_hash = hashlib.sha256(message).hexdigest()
```

```
print("\nOriginal Message:")
```

```
print(message.decode())
```

```
print("\nSimple SHA-256 Hash:")
```

```
print(simple_hash)
```

```
# -----
```

```
# 2. HMAC using SHA-256
```

```
# -----
```

```
hmac_value = hmac.new(
    secret_key,
    message,
    hashlib.sha256
).hexdigest()
```

```
print("\nSecret Key:")
print(secret_key.decode())
```

```
print("\nHMAC-SHA256:")
print(hmac_value)
```

```
# -----
# 3. Verify HMAC
# -----
```

```
received_hmac = hmac_value
```

```
if hmac.compare_digest(hmac_value, received_hmac):
    print("\nHMAC Verification: SUCCESS")
    print("Message integrity and authenticity are verified.")
else:
    print("\nHMAC Verification: FAILED")
```

```
# -----
# 4. Test Modified Message
# -----
```

```
modified_message = b"This is a modified message."
```

```
modified_hmac = hmac.new(
    secret_key,
    modified_message,
    hashlib.sha256
).hexdigest()
```

```
print("\nModified Message:")
```



```

print(modified_message.decode())

print("\nModified Message HMAC:")
print(modified_hmac)

if hmac.compare_digest(hmac_value, modified_hmac):
    print("\nModified Message Verification: SUCCESS")
else:
    print("\nModified Message Verification: FAILED")
    print("The message has been modified.")

# -----
# 5. Security Comparison
# -----

print("\nSECURITY COMPARISON")
print("-" * 60)

print("Simple SHA-256 Hash:")
print("- Does not use a secret key.")
print("- Anyone can calculate the hash.")
print("- Provides integrity checking only.")
print("- Does not authenticate the sender.")

print("\nHMAC-SHA256:")
print("- Uses a secret key.")
print("- Only parties possessing the key can create a valid HMAC.")
print("- Provides message integrity.")
print("- Provides message authentication.")
print("- More secure than using a simple hash for authentication.")

```

Output:

HMAC IMPLEMENTATION

=====

Original Message:

This is an important message.

Simple SHA-256 Hash:

.....

Secret Key:

my-secret-key

HMAC-SHA256:

.....

HMAC Verification: SUCCESS

Message integrity and authenticity are verified.

Modified Message:

This is a modified message.

Modified Message HMAC:

.....

Modified Message Verification: FAILED

The message has been modified.

SECURITY COMPARISON

Simple SHA-256 Hash:

- Does not use a secret key.
- Anyone can calculate the hash.
- Provides integrity checking only.
- Does not authenticate the sender.

HMAC-SHA256:

- Uses a secret key.
- Only parties possessing the key can create a valid HMAC.
- Provides message integrity.
- Provides message authentication.

- More secure than using a simple hash for authentication.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

```
# Simplified Kerberos Authentication Simulation
```

```
# Demonstrates:
```

```
# 1. Ticket generation
```

```
# 2. Ticket distribution
```

```
# 3. Ticket validation
```

```
# 4. Basic replay-attack prevention
```

```
import time
```

```
import secrets
```

```
import hashlib
```

```
import hmac
```

```
# -----
```

```
# Simulated keys shared between trusted components
```

```
# -----
```

```
CLIENT_PASSWORD = "client-password"
```

```
SERVER_SECRET = b"server-secret-key"
```

```
# -----
```

```
# Create a simplified ticket
```

```
# -----
```

```
def generate_ticket(username):
```

```
    timestamp = int(time.time())
```

```
    nonce = secrets.token_hex(8)
```

```
ticket_data = f'{username}}{{timestamp}}{{nonce}'
```

```
# Create a MAC so that the ticket cannot be modified
```

```
ticket_mac = hmac.new(  
    SERVER_SECRET,  
    ticket_data.encode(),  
    hashlib.sha256  
)hexdigest()
```

```
ticket = f'{ticket_data}}{{ticket_mac}'
```

```
return ticket
```

```
# -----
```

```
# Validate the ticket
```

```
# -----
```

```
def validate_ticket(ticket, used_nonces):
```

```
    try:
```

```
        username, timestamp, nonce, received_mac = ticket.split("|")
```

```
        ticket_data = f'{username}}{{timestamp}}{{nonce}'
```

```
        expected_mac = hmac.new(  
            SERVER_SECRET,  
            ticket_data.encode(),  
            hashlib.sha256  
        ).hexdigest()
```

```
        # Check ticket integrity
```

```
        if not hmac.compare_digest(received_mac, expected_mac):  
            return False, "Invalid ticket signature."
```

```
        # Check ticket lifetime
```

```
        current_time = int(time.time())
```

```

        if current_time - int(timestamp) > 60:
            return False, "Ticket expired."

        # Replay attack protection
        if nonce in used_nonces:
            return False, "Replay attack detected."

        used_nonces.add(nonce)

        return True, f"Ticket accepted for user: {username}"

    except Exception:
        return False, "Invalid ticket format."

# -----
# Kerberos Authentication Simulation
# -----

print("SIMPLIFIED KERBEROS AUTHENTICATION")
print("=" * 65)

username = "alice"

print("\nClient:")
print("User:", username)

# Authentication request
print("\nClient -> Authentication Server")
print("Requesting authentication ticket...")

# Ticket generation
ticket = generate_ticket(username)

print("\nAuthentication Server:")
print("Authentication successful.")
print("Ticket generated.")

```

```

# Ticket distribution
print("\nAuthentication Server -> Client")
print("Ticket delivered to client.")

print("\nGenerated Ticket:")
print(ticket)

# -----
# Client sends ticket to application server
# -----

print("\nClient -> Application Server")
print("Sending service ticket...")

used_nonces = set()

# Server validates ticket
valid, message = validate_ticket(ticket, used_nonces)

print("\nApplication Server:")
print(message)

# -----
# Replay Attack Simulation
# -----

print("\nREPLAY ATTACK SIMULATION")
print("=" * 65)

print("\nAttacker captures the previously transmitted ticket.")
print("Attacker attempts to reuse the same ticket...")

valid, message = validate_ticket(ticket, used_nonces)

print("\nApplication Server:")
print(message)

```

```
# -----  
# Security Analysis  
# -----
```

```
print("\nSECURITY ANALYSIS")  
print("-" * 65)
```

```
print("1. The Authentication Server issues a ticket to the client.")  
print("2. The client presents the ticket to the application server.")  
print("3. The server verifies the ticket before granting access.")  
print("4. The ticket contains a timestamp and unique nonce.")  
print("5. The server rejects expired tickets.")  
print("6. The server records used nonces to detect replay attempts.")  
print("7. The ticket is protected using a cryptographic MAC.")  
print("8. A real Kerberos implementation uses stronger and more")  
print("   sophisticated protocols and cryptographic mechanisms.")
```

Output:

SIMPLIFIED KERBEROS AUTHENTICATION

=====

Client:

User: alice

Client -> Authentication Server

Requesting authentication ticket...

Authentication Server:

Authentication successful.

Ticket generated.

Authentication Server -> Client

Ticket delivered to client.

Generated Ticket:

alice|1724...|a83f...|9f2c...

Client -> Application Server

Sending service ticket...

Application Server:

Ticket accepted for user: alice

REPLAY ATTACK SIMULATION

=====

Attacker captures the previously transmitted ticket.

Attacker attempts to reuse the same ticket...

Application Server:

Replay attack detected.

SECURITY ANALYSIS

1. The Authentication Server issues a ticket to the client.
2. The client presents the ticket to the application server.
3. The server verifies the ticket before granting access.
4. The ticket contains a timestamp and unique nonce.
5. The server rejects expired tickets.
6. The server records used nonces to detect replay attempts.
7. The ticket is protected using a cryptographic MAC.
8. A real Kerberos implementation uses stronger and more sophisticated protocols and cryptographic mechanisms.

5. **Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.**


```
# Simplified ElGamal Digital Signature
# Demonstrates key generation, signing and verification.
#
# This is an educational implementation.
# Real applications should use a vetted cryptographic library.
```

```
import hashlib
import secrets
import math
```

```
# -----
# Modular inverse
# -----
```

```
def mod_inverse(a, m):
    return pow(a, -1, m)
```

```
# -----
# ElGamal parameters
# Small prime is used only for demonstration.
# -----
```

```
p = 467
g = 2
```

```
# -----
# Key Generation
# -----
```

```
private_key = secrets.randbelow(p - 2) + 1
public_key = pow(g, private_key, p)
```

```
print("ELGAMAL DIGITAL SIGNATURE")
print("=" * 65)
```

```
print("\nPublic Parameters:")
```

```
print("Prime (p):", p)
print("Generator (g):", g)
```

```
print("\nPrivate Key:")
print(private_key)
```

```
print("\nPublic Key:")
print(public_key)
```

```
# -----
# Message
# -----
```

```
message = b"This is a secure message."
```

```
print("\nOriginal Message:")
print(message.decode())
```

```
# -----
# Hash the message
# -----
```

```
message_hash = int(
    hashlib.sha256(message).hexdigest(),
    16
) % (p - 1)
```

```
print("\nMessage Hash:")
print(message_hash)
```

```
# -----
# Signature Generation
# -----
```

```

# Choose random k such that gcd(k, p-1) = 1
while True:
    k = secrets.randbelow(p - 2) + 2
    if math.gcd(k, p - 1) == 1:
        break

r = pow(g, k, p)

k_inverse = mod_inverse(k, p - 1)

s = ((message_hash - private_key * r) * k_inverse) % (p - 1)

signature = (r, s)

print("\nRandom Signing Value (k):")
print(k)

print("\nDigital Signature:")
print("r =", r)
print("s =", s)

# -----
# Signature Verification
# -----

left_side = (
    pow(public_key, r, p) *
    pow(r, s, p)
) % p

right_side = pow(
    g,
    message_hash,
    p
)

```

```
print("\nSignature Verification")
print("-" * 65)
```

```
print("Left Side :", left_side)
print("Right Side:", right_side)
```

```
if left_side == right_side:
    print("\nSignature Verification: SUCCESS")
    print("The signature is valid.")
else:
    print("\nSignature Verification: FAILED")
    print("The signature is invalid.")
```

```
# -----
# Modified Message Test
# -----
```

```
modified_message = b"This message has been modified."
```

```
modified_hash = int(
    hashlib.sha256(modified_message).hexdigest(),
    16
) % (p - 1)
```

```
modified_right_side = pow(
    g,
    modified_hash,
    p
)
```

```
print("\nModified Message Test")
print("-" * 65)
```

```
if left_side == modified_right_side:
    print("Modified Message Verification: SUCCESS")
else:
```

```

print("Modified Message Verification: FAILED")
print("Modification detected.")

# -----
# Security Analysis
# -----

print("\nSECURITY ANALYSIS")
print("-" * 65)

print("1. ElGamal uses a private key to generate the signature.")
print("2. The corresponding public key is used for verification.")
print("3. The message is hashed before signing.")
print("4. Any modification to the message changes its hash.")
print("5. A modified message therefore fails signature verification.")
print("6. Security depends on the difficulty of the discrete logarithm problem.")
print("7. A fresh secure random signing value must be used for every signature.")
print("8. The small parameters used here are for education only.")
print("9. Real systems should use established cryptographic libraries.")

```

Output:

ELGAMAL DIGITAL SIGNATURE

=====

Public Parameters:

Prime (p): 467

Generator (g): 2

Private Key:

123

Public Key:

125

Original Message:

This is a secure message.

Message Hash:

...

Random Signing Value (k):

...

Digital Signature:

r = ...

s = ...

Signature Verification

Left Side : ...

Right Side: ...

Signature Verification: SUCCESS

The signature is valid.

Modified Message Test

Modified Message Verification: FAILED

Modification detected.

SECURITY ANALYSIS

1. ElGamal uses a private key to generate the signature.
2. The corresponding public key is used for verification.
3. The message is hashed before signing.
4. Any modification to the message changes its hash.
5. A modified message therefore fails signature verification.
6. Security depends on the difficulty of the discrete logarithm problem.
7. A fresh secure random signing value must be used for every signature.
8. The small parameters used here are for education only.

